

Lecture 02 Handout — The Concept of Operations and the Process Documents for Operations

Software-engineering module, Lecture 02. Read before the second meeting (ten minutes). Keep it open during the demo.

This handout reproduces the files that Lecture 02 adds to the repository as Lecture 01 left it. They are copied from `demos/lecture-01-note-specs-demo/starter-102/` in the course repository, which also contains the Lecture 01 end state (the specification at 1.0.0, one note, the index, and the verifier); only the heading levels are shifted.

File	Role in the lecture
<code>note-set-conops.md</code>	The concept of operations: what the set is for, who acts, three scenarios in the user's words, and the three operations they become, each with a precondition and a postcondition. Version 0.1, a draft; the demo audits it.
<code>CLAUDE.md</code>	The loader, now naming the concept of operations as the first governing document and five process documents.
<code>process/README.md</code>	The invariant the process documents exist to keep true, and a map of the files.
<code>process/development-rules.md</code>	How an operation begins and ends, what may change and with whose approval, what is written down instead of guessed: DEV-1 to DEV-7. Always loaded.
<code>process/conops-audit.md</code>	Audit rules that apply only to a concept of operations: AUDCON-1 to AUDCON-6. Run with the specification audit when the ConOps is audited.
<code>BACKLOG.md</code>	Where a question the operation cannot settle is written (DEV-6).

The three process documents from Lecture 01 are also present, each at version 1.1: the specification audit gains the concept of operations in its binding table; the reporting document gains the operation completion note (RPT-5); the verification document gains when verification runs (VER-3) and the gate (VER-4).

File: `note-set-conops.md` — the concept of operations.

Note Set — Concept of Operations

Version: 0.1 (draft) Status: draft for review; not yet realized.

1. Purpose and scope

A single user keeps a personal set of study notes on software-engineering classics. A coding agent performs all structural operations on the note set; the user writes and edits note content. This document describes the note set and the operations the agent supports, from the user's point of view.

2. The note set

Notes are markdown files stored in a `notes/` directory inside a git repository. An index file, `index.md`, at the top level of the repository links to every note in the set. The formatting of markdown files in the note set is governed by `note-format-spec.md`.

3. Roles

There is one user and one agent (Claude Code). The user requests operations conversationally; the agent performs them and reports what changed.

4. Scenarios and agent operations

Scenarios Three sessions, as the user would describe them.

- **S1 — A placeholder.** “I want a note for a paper I have not read yet, so that it is in the index and I remember to come back to it. I give the agent the title and nothing else.”
- **S2 — Notes from reading.** “I have just read Brooks’s *No Silver Bullet* and typed a page of notes with a few headings. I paste them and ask the agent to add a note. Later I open the index and see the note listed.”
- **S3 — A mistake.** “That note was a mistake. I tell the agent the title and ask it to get rid of it; afterwards the index should not mention it.”

O1 — Add blank note

- **Input:** a title.
- **Precondition:** no note with that title exists in the set.
- **Postcondition:** a new note exists containing well-formed front-matter and the title as its heading, with no other content; `index.md` lists it.

O2 — Add note with initial content

- **Inputs:** a title and initial markdown text.
- **Precondition:** no note with that title exists in the set.
- **Postcondition:** a note exists whose content is the supplied text brought into conformance with `note-format-spec.md`, preserving the user’s meaning; `index.md` lists it.

O3 — Remove note

- **Input:** a title.
- **Precondition:** a note with that title exists in the set.
- **Postcondition:** the note file no longer exists; `index.md` no longer lists it.

5. Conformance

Every note in the set is expected to conform to `note-format-spec.md` at all times — conformance is an invariant of the set, not a milestone. The user may at any time direct the agent to check one note, or all notes, for conformance. A conformance report cites the specific rule identifiers (R1, R2, ...) from the format spec.

6. Change management

Every operation ends in a git commit. The specifications themselves live in the same repository and are maintained documents: when they change, they change by versioned, reviewed edits.

File: CLAUDE.md — the loader.

Note set — agent orientation

What this repository is

A set of markdown study notes maintained by an agent that follows written specifications. The concept of operations says what the set is for, who uses it, and which operations the agent performs; the format

specification says what a conformant note is. Conformance is kept as an invariant while the set changes.

Governing documents, in authority order

1. `note-set-conops.md` — purpose, roles, the agent's operations (O1–O3) with their preconditions and postconditions, conformance, change management.
2. `note-format-spec.md` — what a conformant note is: the rules R1..., at the version its changelog states.
3. `note-set-operations.md` — how the agent carries out each operation. Derived from 1 and 2 and never contradicting them (DEV-5); created by this project's first operation.

Process — how work proceeds with respect to the specifications

`process/README.md` states the invariant and maps the files.

The rules in the following documents are always in effect (always on) (loaded with this file):
`@process/development-rules.md` `@process/reporting.md` `@process/verification.md`

The rules in the following documents are only invoked when needed (on demand) — run before any plan, after any changes to a specification, or when asked: `process/spec-audit.md` (AUD) for any specification; `process/conops-audit.md` (AUDCON) in addition, for the concept of operations.

Bootstrap rules

1. Read the governing documents and the always-on process documents, in full, before any operation.
2. Never modify a governing document or a process document without explicit approval; propose changes as an amendment proposal (RPT-3) and wait (DEV-3).

Commands

- `python3 check_notes.py --all` — the algorithmic verifier in whole-set mode; the gate (VER-4), once it exists.

File: `process/README.md` — the invariant and the map.

Process — how development proceeds with respect to the specifications

Version: 1.0 Status: normative; maintained.

The invariant

Conformance is an invariant of the repository, not a milestone. At every commit:

1. every specification passes its audit — AUD for any specification, and AUDCON in addition for the concept of operations;
2. every realization conforms to the current version of its specification — VER;
3. every change to either side is recorded with the decision that caused it — DEV, RPT.

The specifications say what the system is. The documents in this folder say how work proceeds with respect to them. Both are specifications: versioned, audited, cited by identifier. The realization of *these* documents is the repository's history — commits, changelog entries, reports — and auditing the process means reading that history.

The files

File	Concern	Prefix	Applies	Loaded
<code>development-rules.md</code>	how an operation begins, proceeds, and ends; who may change what	DEV	every operation	always
<code>reporting.md</code>	the forms of a gap list, a conformance report, an amendment proposal, a completion note	RPT	whenever something is reported	always
<code>verification.md</code>	the kinds of verifier; mechanical vs judgment; when checks run; the gate	VER	after every change; before every commit	always
<code>spec-audit.md</code>	how a specification's quality is assessed	AUD	before a plan; after an amendment; on request	on demand
<code>conops-audit.md</code>	rules specific to a concept of operations	AUDCON	with AUD, whenever the ConOps is audited	on demand

Always on: the rules are always in effect — the file is loaded with `CLAUDE.md`. *On demand:* the rules are invoked only when needed — before any plan, after any changes to a specification, or when asked.

How they fit together

An operation begins by reading (DEV-1). If it will change a specification, the change is proposed (RPT-3), approved (DEV-3), recorded, and the amended document is audited again (AUD; AUDCON for the ConOps). If it changes a realization, the realization is verified (VER-3, VER-4) and the result reported (RPT-2). A question the operation cannot settle goes to `BACKLOG.md` (DEV-6). The operation ends with a completion note (RPT-5) and a commit named for it (DEV-7).

Changing these documents

Process documents change by the same mechanism as specifications: an amendment proposal (RPT-3), approval (DEV-3), a version bump, and a changelog entry. Rule identifiers are never renumbered; new rules are appended.

Changelog

- **1.0** (2026-09-19) — Initial.

File: `process/development-rules.md` — the development rules (DEV).

Development rules (DEV)

Version: 1.1 Status: normative; maintained. **Always on.**

This document specifies how development proceeds with respect to the specifications: how an operation begins, what may change and with whose approval, what is written down instead of guessed, and how an operation ends. The keywords **MUST**, **MUST NOT**, **SHOULD**, and **MAY** carry their RFC 2119 meanings.

In this repository

An *operation* is one of the agent operations the concept of operations promises — O1 add blank note, O2 add note with initial content, O3 remove note, and the conformance check — or a change to a specification, or a change to a process document. Governing documents, in authority order: `note-set-conops.md`, `note-format-spec.md`; derived: `note-set-operations.md`. Deferred questions go to `BACKLOG.md`.

Rules

DEV-1 — Read before acting Before any operation the agent **MUST** read the governing documents and the always-on process documents in full, from the files — not from its recollection of an earlier reading. A check driven by memory is one of the ways a verification result becomes invalid (VER-1).

DEV-2 — Specification before realization A realization changes for one of three reasons, and the history **MUST** show which.

- (a) **Repair.** The realization did not conform and is changed so that it does. No specification change precedes it; a report (RPT-2) and a ruling (DEV-4, RPT-4) do.
- (b) **Conformance-preserving change.** The realization conformed before and conforms after; what changed lies in what the specification deliberately leaves open — the algorithm, its efficiency, the structure or naming of the code. No specification change is needed. The change **MUST** be verified (VER-3), and its commit message **MUST** state that no specified behavior changed.
- (c) **Change of specified behavior.** The realization would no longer conform to the current specification. Such a change **MUST** trail a committed amendment to the specification (DEV-3, RPT-3) — in the same commit or an earlier one. The intent changes first; the realization follows.

A change of kind (b) motivated by a property the developer would not do without — a bound on time or size, a portability constraint — is a sign that the specification is incomplete at its own level (AUD-4): the property is added by amendment, after which kind (c) governs it, or deferred explicitly (DEV-6).

Test: for each commit that touches a realization — which kind is it, and does the history show what that kind requires?

Example (a): the careless Royce note violated R2–R5 and was repaired after the report and the ruling; the specification did not move.

Example (b): `check_notes.py` is restructured so that each rule is one function; its output on every note is unchanged, the gate is green, and the commit says so.

Example (c): the scope amendment (“`index.md` is not a note”) and the filename rule were committed before the scaffold that created `index.md` and the first note.

DEV-3 — Amend only with approval No governing document and no process document **MUST** be edited without explicit approval. Changes are proposed as an amendment proposal (RPT-3); each approved amendment bumps the document’s version and adds a changelog entry stating the rationale.

DEV-4 — In a verification failure, a person decides whether the specification or realization gets changed When a specification and a realization disagree, the finding is a fact about the pair; which side changes is the developer’s decision. The decision and its reason **MUST** be recorded — in the changelog when the specification moves, in the commit message when the realization moves.

Example: a note’s H1 disagreed with its `title` field. The ruling — the field is authoritative; repairs adjust the heading — was written into R3 so that nobody has to decide it case by case.

DEV-5 — Derived documents follow governing ones A document derived from the governing specifications — here `note-set-operations.md` — MUST NOT contradict them. When it is wrong or incomplete, the agent proposes a correction to it rather than improvising; when it and a governing document disagree, the governing document wins and the derived document is corrected.

Example: the operations document says so itself: “If this document disagrees with either specification, the specifications win and this document must be corrected.”

DEV-6 — Defer explicitly A question the operation cannot settle MUST be written to `BACKLOG.md` — the question, where it arose, the options if known — and MUST NOT be settled by a silent guess. A deferred question is closed by a ruling, recorded where DEV-4 says.

Example: “O1 creates blank notes with a placeholder Summary to satisfy R8’s MUST clause. Should R8 instead exempt notes that have no content yet?” — deferred, with the current practice stated alongside.

DEV-7 — Every operation ends the same way An operation is complete only when the affected realizations have been verified (VER-3, VER-4) and the result reported (RPT-2); a completion note has been given (RPT-5); and a commit has been made whose message names the operation.

Example: `add-note-with-content: No Silver Bullet.`

Changelog

- **1.1** (2026-09-19) — DEV-2 restated as three kinds of realization change — repair, conformance-preserving, change of specified behavior — with what the history must show for each.
- **1.0** (2026-09-19) — Initial: DEV-1 through DEV-7.

File: `process/conops-audit.md` — the concept-of-operations audit (AUDCON).

Concept-of-operations audit (AUDCON)

Version: 1.0 Status: normative; maintained. **Invoked on demand**, together with `process/spec-audit.md`.

A concept of operations (ConOps) is a specification of purpose: what the system is for, who uses it, which operations they perform, and what they can observe. It is audited by `process/spec-audit.md` (AUD-1 through AUD-7) like any specification, and by the additional rules below, which apply only to this kind of document. The output is one gap list per RPT-1 covering both the AUD and the AUDCON findings. The keywords MUST, MUST NOT, SHOULD, and MAY carry their RFC 2119 meanings.

Two skeletons are in use in this course. The **light** skeleton: purpose and scope; the system; roles; operations with preconditions and postconditions; conformance; change management. The **full** skeleton: nine numbered sections, of which 2 (Current Situation), 3 (Justification for and Nature of the Changes), and 6 (Summary of Impacts) apply only when an existing system is being improved. The rules below are written for both; rules specific to the full skeleton are added when it is first applied.

In this repository

Under audit	Skeleton	Cited as
<code>note-set-conops.md</code>	light	§1 ... §6; operations O1, O2, O3

Rules

AUDCON-1 — Implementation-independent The ConOps MUST NOT name anything that exists only inside the realization: no programming language, module, class, method, data structure, or internal file.

What the user sees, types, opens, or is told — a markdown file in `notes/`, a command, a report that cites rule identifiers — is operational and belongs.

Test: for each concrete noun, ask whether a user of the system encounters it. If only a builder would, it is misplaced.

Example: “notes are markdown files in `notes/`” belongs. “`check_notes.py` parses the front-matter as two scalar fields” belongs in a lower-level document.

AUDCON-2 — Observable Every policy the ConOps states **MUST** be something a user could observe, or an operation could demonstrate.

Test: for each policy, name the moment at which a user would notice it being kept or broken.

Example: “conformance is an invariant of the set, not a milestone” is observable — a check requested at any moment passes. “The agent reads the specifications before every operation” is not a policy of the system but a rule of the process (DEV-1), and belongs there.

AUDCON-3 — Operations and scenarios cover Every operation the ConOps promises **MUST** state its input, its precondition (what must hold before it may start), and its postcondition (what holds when it is done), from the user’s side. Every mode, policy, and user class the ConOps names **MUST** appear in at least one operation or scenario.

Test: list the operations; for each, can a user tell whether it may start and whether it finished correctly? List the user classes and policies; is each exercised somewhere?

Example: O3 (remove note) requires “a note with that title exists” — a precondition a user can check — and its postcondition names both the file and the index entry.

AUDCON-4 — Terms Every term the ConOps uses repeatedly **MUST** be defined — in a glossary, or in the text where it first appears — and every defined term **MUST** be used with that meaning throughout, consistently with the other specifications (AUD-3).

Example: “note”, “index”, and “the set” are defined in §2; “conformance” is used in the sense `note-format-spec.md` gives it.

AUDCON-5 — Voice and self-containment The ConOps **MUST** be written in the third person and the present tense, and **MUST** be understandable by a reader who has no other document. It **MUST NOT** cite code. It **SHOULD NOT** depend on clauses of lower-level specifications; naming a lower-level document as the place where a detail is fixed is acceptable.

Example: “a conformance report cites the rule identifiers (R1, R2, ...) of the format specification” names the document and describes what the user sees — a **SHOULD**-level finding that a ruling may close as *no change: observable behavior*.

AUDCON-6 — Versioned The ConOps **MUST** carry a version, a status, and a changelog whose entries state the rationale for each change (DEV-3).

Changelog

- **1.0** (2026-09-19) — Initial: AUDCON-1 through AUDCON-6.

File: BACKLOG.md — deferred questions.

Backlog — deferred questions and loose ends

Per DEV-6: a question an operation cannot settle is written here, never settled by a silent guess. An entry gives the date, where the question arose (a document and clause, or an operation), the question, the options if known, and its status. A deferred question is closed by a ruling, recorded where DEV-4 says (the changelog when a specification moves; the commit message when a realization moves), and the entry is marked *decided* with a pointer to that record.

Deferred questions

(none yet)

Loose ends

(none yet)